WEEK-7

Simulate frame allocation Methods a)minimum number of frames b) equal allocation c) proportional allocation

AIM- To Implement the frame allocation methods such as minimum number of frames and equal allocation and proportional allocation.

ALGORITHM-

MINIMUM NUMBER:

- Start.
- Read the number of processes.
- Read the total number of available frames.
- Read the minimum frames required by each process.
- Check whether the sum of minimum frames is less than or equal to the available frames.
- If yes, allocate the minimum frames to every process.
- If no, display **Allocation Not Possible**.
- Display the allocation.
- Stop.

EQUAL ALLOCATION:

- Start.
- Read number of processes.
- Read total number of frames.
- Calculate framesPerProcess = totalFrames / numberOfProcesses.
- Allocate the same number of frames to every process.
- Calculate remaining frames.
- Display the allocation.
- Stop.

PROPORTIONAL ALLOCATION

- Start.
- Read the number of processes.
- Read the total number of frames.
- Read the size of each process.
- Calculate the total size of all processes.

- For every process, calculate:
  allocation = (process size / total size) × total frames.
- Convert the result to an integer for frame allocation.
- Display the number of frames allocated to each process.
- Stop.

CODE:

```
#include <stdio.h>


int main()
{
   int n, totalFrames;
   int minFrames[20];
   int i, sum = 0;


   printf("=====================================\n");
   printf("   MINIMUM NUMBER OF FRAMES\n");
   printf("=====================================\n");


   printf("Enter number of processes: ");
   scanf("%d", &n);


   printf("Enter total number of frames: ");
   scanf("%d", &totalFrames);


   printf("Enter minimum frames required by each process:\n");


   for(i = 0; i < n; i++)
   {
      printf("P%d: ", i + 1);
      scanf("%d", &minFrames[i]);
      sum += minFrames[i];
```

```
    }

    if(sum > totalFrames)
    {
        printf("\nAllocation Not Possible!\n");
    }
    else
    {
        printf("\nFrame Allocation:\n");

        for(i = 0; i < n; i++)
        {
            printf("P%d -> %d frames\n", i + 1, minFrames[i]);
        }

        printf("Frames Used = %d\n", sum);
        printf("Frames Remaining = %d\n", totalFrames - sum);
    }

    return 0;
}
```

VNR VJIET

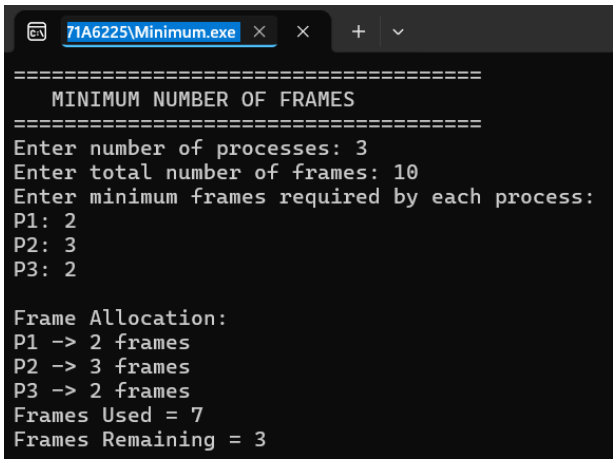
Name of the Laboratory:

Name of the Experiment:

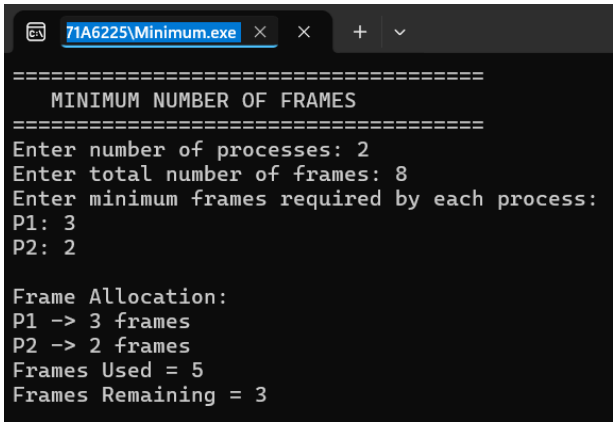
Experiment No: **Date:**

OUTPUT:

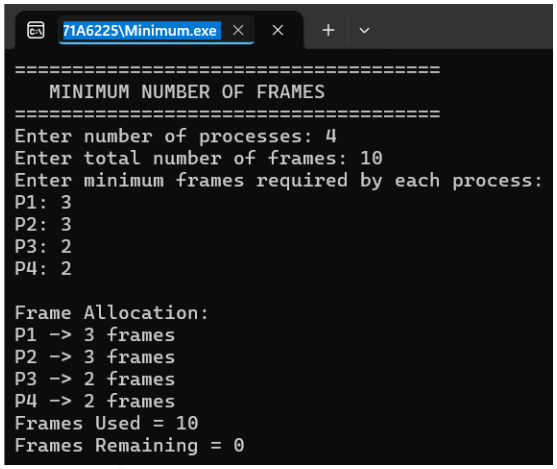
TESTCASE 1:



TEST CASE 2:



TEST CASE 3:



CODE:

```
#include <stdio.h>
int main()
{
   int n, totalFrames;
   int framesPerProcess, remaining;
   int i;
   printf("        EQUAL ALLOCATION\n");
   printf("=====================================\n");

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter total number of frames: ");
   scanf("%d", &totalFrames);

   framesPerProcess = totalFrames / n;
   remaining = totalFrames % n;

   printf("\nFrame Allocation:\n");

 for(i = 0; i < n; i++)
   {
      printf("P%d -> %d frames\n", i + 1, framesPerProcess);
   }
printf("Frames Remaining = %d\n", remaining);
return 0;
}
```

VNR VJIET

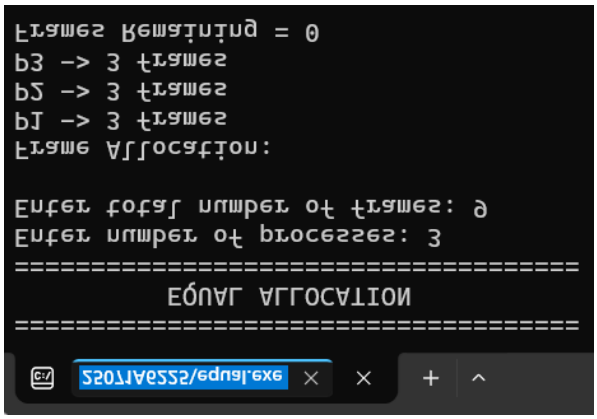
Name of the Laboratory:

Name of the Experiment:_____

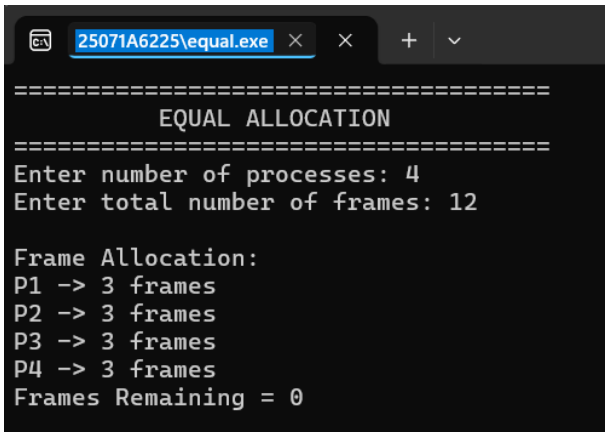
Experiment No:_____ **Date:**_____

OUTPUT:

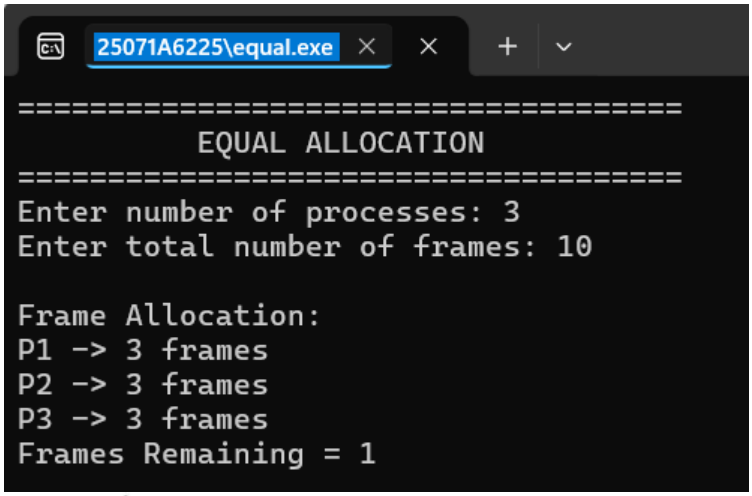
TESTCASE 1:



TEST CASE 2:



TEST CASE 3:



CODE:

```
#include <stdio.h>


int main()
{
   int n, totalFrames;
   int size[20], allocation[20];
   int i, totalSize = 0;
   int allocated = 0;


   printf("=====================================\n");
   printf("     PROPORTIONAL ALLOCATION\n");
   printf("=====================================\n");


   printf("Enter number of processes: ");
   scanf("%d", &n);


   printf("Enter total number of frames: ");
   scanf("%d", &totalFrames);


   printf("Enter size of each process:\n");


   for(i = 0; i < n; i++)
   {
      printf("P%d: ", i + 1);
      scanf("%d", &size[i]);
      totalSize += size[i];
   }
```

```
    printf("\nFrame Allocation:\n");

    for(i = 0; i < n; i++)
    {
        allocation[i] =
            (size[i] * totalFrames) / totalSize;

        allocated += allocation[i];

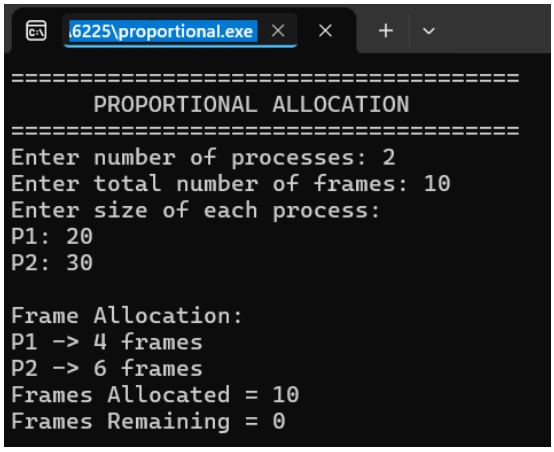
        printf("P%d -> %d frames\n",
                i + 1, allocation[i]);
    }

    printf("Frames Allocated = %d\n", allocated);
    printf("Frames Remaining = %d\n",
            totalFrames - allocated);

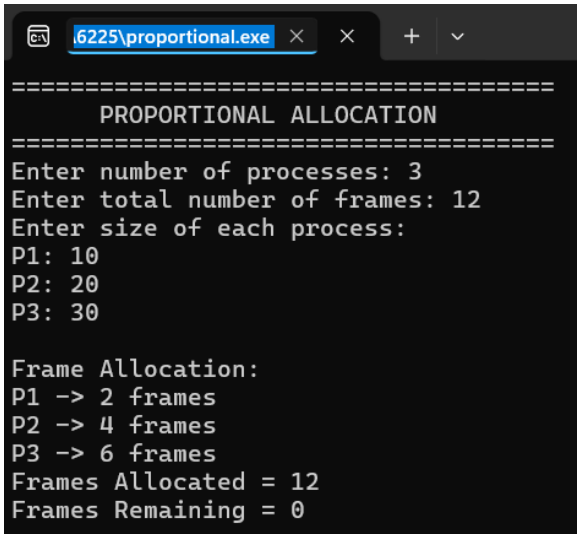
    return 0;
}
```

OUTPUT:

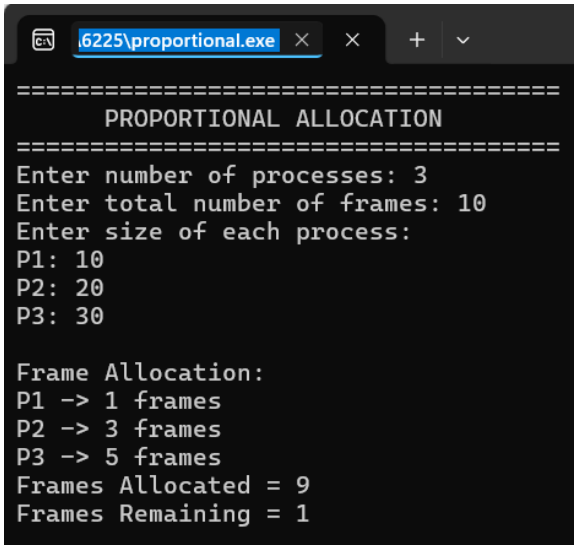
TESTCASE 1:



TEST CASE 2:



TEST CASE 3;



WEEK-10

Simulate all file allocation strategies. a)Sequential b) Indexed c) Linked

AIM- To Implement the file allocation methods such as sequential allocation and indexed allocation and linked allocation.

ALGORITHM-

SEQUENTIAL ALLOCATION:

- Start.
- Read the starting block and number of blocks required.
- Allocate consecutive blocks starting from the starting block.
- Display all allocated blocks.
- Stop.

INDEXED ALLOCATION:

- Start.
- Read the index block.
- Read the number of file blocks.
- Read the block numbers to be stored in the index.
- Store the block numbers in the index block.
- Display the index block and allocated blocks.
- Stop.

LINKED ALLOCATION:

- Start.
- Read the number of blocks.
- Read the block numbers.
- Link each block to the next block.
- Set the last block's pointer to NULL.
- Display the linked sequence.
- Stop.

CODE:

```
#include <stdio.h>

int main()
{
   int start, length;
   int i;

   printf("=======================================\n");
   printf("       SEQUENTIAL FILE ALLOCATION\n");
   printf("=======================================\n");

   printf("Enter starting block: ");
   scanf("%d", &start);

   printf("Enter number of blocks: ");
   scanf("%d", &length);

   printf("\nAllocated Blocks:\n");

   for(i = 0; i < length; i++)
   {
      printf("%d ", start + i);
   }

   printf("\n");

   return 0;
}
```

VNR VJIET

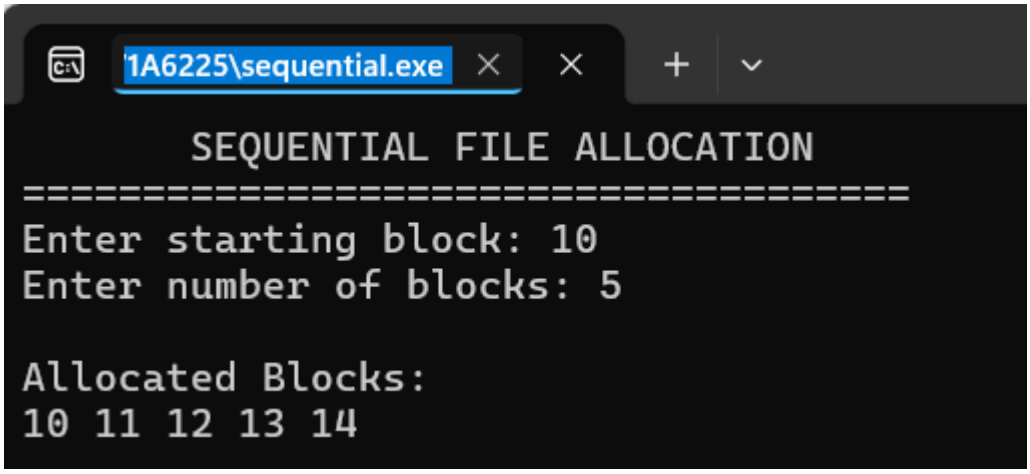
Name of the Laboratory:

Name of the Experiment:

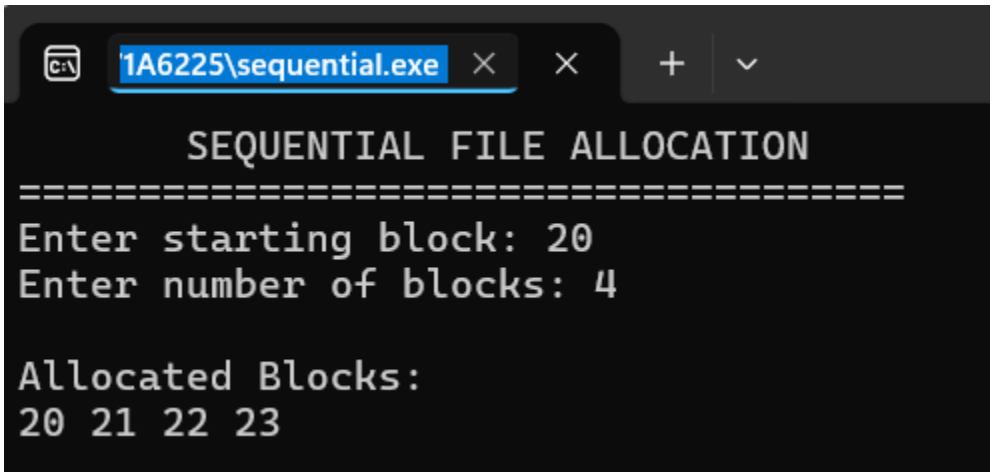
Experiment No: **Date:**

OUTPUT:

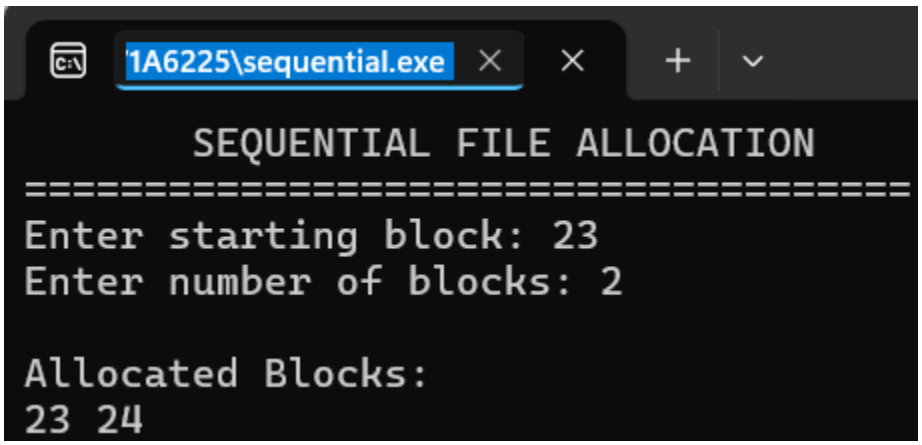
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:



CODE:

```
#include <stdio.h>


int main()
{
   int indexBlock;
   int blocks[20];
   int n, i;
   printf("         INDEXED FILE ALLOCATION\n");
   printf("=======================================\n");

   printf("Enter index block: ");
   scanf("%d", &indexBlock);

   printf("Enter number of blocks: ");
   scanf("%d", &n);

   printf("Enter block numbers:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &blocks[i]);
   }
       printf("\nIndex Block = %d\n", indexBlock);
       printf("Blocks allocated through index block:\n");
          for(i = 0; i < n; i++)
                    printf("%d ", blocks[i]);
           printf("\n");
    return 0;
}
```

VNR VJIET

Name of the Laboratory:

Name of the Experiment:

Experiment No: **Date:**

OUTPUT:

TEST CASE 1:

```
071A6225\indexed.exe
        INDEXED FILE ALLOCATION
====================================
Enter index block: 2
Enter number of blocks: 4
Enter block numbers:
7 12 18 25

Index Block = 2
Blocks allocated through index block:
7 12 18 25
```

TEST CASE 2:

```
071A6225\indexed.exe
        INDEXED FILE ALLOCATION
====================================
Enter index block: 5
Enter number of blocks: 3
Enter block numbers:
10 20 30

Index Block = 5
Blocks allocated through index block:
10 20 30
```

TEST CASE 3:

```
071A6225\indexed.exe
        INDEXED FILE ALLOCATION
====================================
Enter index block: 4
Enter number of blocks: 3
Enter block numbers:
21 12 22

Index Block = 4
Blocks allocated through index block:
21 12 22
```

CODE:

```
#include <stdio.h>


int main()
{
   int blocks[20];
   int n, i;
   printf("        LINKED FILE ALLOCATION\n");
   printf("=====================================\n");
  printf("Enter number of blocks: ");
   scanf("%d", &n);
   printf("Enter block numbers:\n");
  for(i = 0; i < n; i++)
   {
      scanf("%d", &blocks[i]);
}
printf("\nLinked Allocation:\n");


   for(i = 0; i < n; i++)
   {printf("%d", blocks[i]);
    if(i < n - 1)
        printf(" -> ");
      else
        printf(" -> NULL");
   }
printf("\n");
return 0;
}
```

VNR VJIET

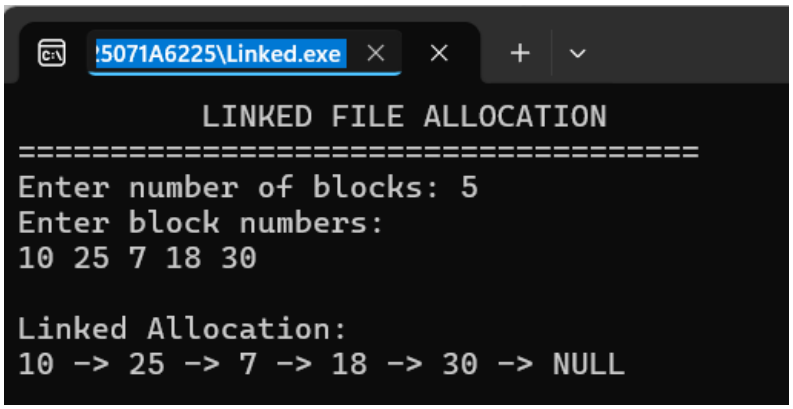
Name of the Laboratory: _____

Name of the Experiment: _____

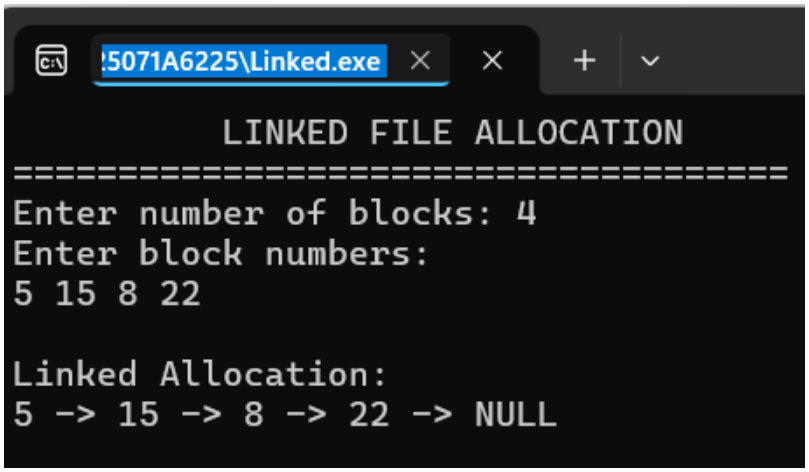
Experiment No: _____ **Date:** _____

OUTPUT:

TEST CASE 1:



TEST CASE 2:



TEST CASE 3:

